PROJEKTDOKUMENTATION

WLAN-Access-Point · Routing · Firewall

Konfiguration, Nachweis & automatisiertes Setup-Skript

Parameter	Wert
Hostname	WLAN-AP-Router
Hardware	Raspberry Pi 5 · 2 GB RAM
Betriebssystem	Raspberry Pi OS Lite (64-Bit) – Headless
Zeitzone / Locale	Europe/Berlin · Tastatur: de
Benutzername	projekt_user
WLAN-SSID	Production
WLAN-Passwort	Production-01
IoT-Netz (wlan0)	192.168.50.0/24 · Gateway 192.168.50.1
DHCP-Bereich	192.168.50.20 – 192.168.50.150 · Lease 24 h
DNS-Domain	lab.local · Upstream: 8.8.8.8
LAN (eth0)	192.168.178.69 · Gateway 192.168.178.1

1. Einleitung und Systemüberblick

Dieses Dokument beschreibt die vollständige Konfiguration des WLAN-AP-Routers im Rahmen des Hochschulprojekts "Angewandtes Informatik Projekt" (Kapitel 2: Deployment of the Core Infrastructure, Prof. Dr. J. Schneider). Als Hardware-Plattform dient ein Raspberry Pi 5 mit 2 GB RAM, auf dem Raspberry Pi OS Lite (64-Bit) in einem Headless-Setup betrieben wird.

Das System vereint drei Kernfunktionen:

- WLAN-Access-Point: Das Produktionsnetzwerk "Production" (IEEE 802.11ac, WPA2) wird über das Interface wlan0 bereitgestellt. IoT-Geräte (z. B. Raspberry Pi Pico W) und weitere Clients verbinden sich hierüber.
- Routing & NAT: Der Linux-Kernel leitet Datenpakete zwischen wlan0 (192.168.50.0/24) und eth0 (192.168.178.0/24) weiter. Masquerading ermöglicht den Internet-Zugriff aller WLAN-Clients.
- Firewall (nftables): Eine Stateful-Firewall mit Default-Deny-Policy schützt das System nach den Sicherheitsprinzipien aus den Vorlesungsunterlagen.

> *ℹ Abweichung von der Musterlösung: In der Firewall wurde zusätzlich SSH-Zugriff über eth0 (iif "eth0" tcp dport 22 accept) erlaubt. Die Begründung dafür findet sich in Abschnitt 5.4.*

2. Hardware und Betriebssystem-Installation

2.1 Hardware

Komponente	Spezifikation
Prozessor	Quad-Core ARM Cortex-A76, 64-Bit, bis 2,4 GHz
RAM	2 GB SDRAM
Speicher	Micro-SD 32 GB
Ethernet	1× Gigabit (eth0)
WLAN	Dual-Band 802.11ac – 2,4 / 5,0 GHz (wlan0)
Stromversorgung	USB-Type-C @ 5 V / 3 A
Betriebssystem	Raspberry Pi OS Lite 64-Bit (Bookworm)

2.2 OS-Installation mit Raspberry Pi Imager

Das Betriebssystem wurde mit dem Raspberry Pi Imager auf die Micro-SD-Karte geschrieben. Folgende Optionen wurden bereits im Imager-Wizard konfiguriert (Headless-Setup – kein Bildschirm nötig):

Einstellung	Gewählter Wert	Begründung
OS	Raspberry Pi OS Lite (64-Bit)	Kein Desktop → minimale Angriffsfläche (Least Common Mechanism)
Hostname	WLAN-AP-Router	Identifikation im Netzwerk
SSH aktiviert	Ja	Headless-Fernzugriff ohne Bildschirm
Benutzername	projekt_user	Kein Standard-Name (admin, pi) → Sicherheit
Passwort	Sicherheit2026!	> 8 Zeichen, Zahl + Sonderzeichen
Zeitzone	Europe/Berlin	Korrekte Zeitstempel in Logs
Tastaturlayout	de (Deutsch)	Korrekte Eingabe von Sonderzeichen
WLAN-Land	DE	Korrekte Frequenzkanäle (gesetzlich)

Nach dem ersten Boot: Systempakete aktualisieren:

```
sudo apt update && sudo apt upgrade -y
```

3. WLAN-Access-Point

3.1 Konzept

Die gesamte Netzwerkkonfiguration wird über den **NetworkManager** verwaltet. Für den WLAN-Access-Point findet der Linux-Kernel-Treiber mac80211 Verwendung, während der hostapd-Daemon die WPA2-Authentifizierung im Hintergrund übernimmt. Da der NetworkManager diese Prozesse intern steuert, ist die Erstellung einer separaten hostapd.conf nicht erforderlich.

Parameter	Wert
Interface	wlan0
SSID	Production
Sicherheitsstandard	WPA2 Personal (PSK)
WLAN-Passwort	Production-01
Gateway-IP (AP)	192.168.50.1 / 24
DHCP-Bereich	192.168.50.20 – 192.168.50.150
DNS-Domain (lokal)	lab.local
Lease-Time	24 Stunden
Upstream-DNS	8.8.8.8 (Google DNS)

3.2 Installation

```
sudo apt install hostapd dnsmasq -y
```

3.3 Konfiguration des Access-Points (nmcli)

Anlegen der WLAN-AP-Verbindung via NetworkManager:

```
sudo nmcli connection add type wifi ifname wlan0 con-name ap-wlan0 mode ap ssid Production
```

WLAN-Passwort setzen (WPA2):

```
sudo nmcli connection modify ap-wlan0 wifi-sec.psk "Production-01"
```

AP aktivieren:

```
sudo nmcli connection up ap-wlan0
```

Status und verbundene Clients prüfen:

```
sudo nmcli device status
sudo iw dev wlan0 station dump
```

3.4 DHCP- und DNS-Server (dnsmasq)

Backup der Standardkonfiguration, dann eigene Konfiguration:

```
sudo mv /etc/dnsmasq.conf /etc/dnsmasq.conf.backup
```

Inhalt der Datei /etc/dnsmasq.conf:

```
# Interface, auf dem dnsmasq läuft
interface=wlan0
bind-interfaces

# Lokale DNS-Domain
domain=lab.local
local=/lab.local/

# DHCP-Bereich  (Lease: 24 h)
dhcp-range=192.168.50.20,192.168.50.150,255.255.255.0,24h

# Default-Gateway für WLAN-Clients
dhcp-option=3,192.168.50.1

# DNS-Server für WLAN-Clients  (= dieser Raspberry Pi)
dhcp-option=6,192.168.50.1

# Upstream-DNS (Google)
server=8.8.8.8
```

Dienst dauerhaft aktivieren und starten:

```
sudo systemctl enable dnsmasq
sudo systemctl start dnsmasq
```

Verifikation (Systemausgabe, 13 h Laufzeit):

```
● dnsmasq.service  Active: active (running) since Sat 2026-04-18 23:51:21 CEST
  DHCP, IP range 192.168.50.20 -- 192.168.50.150, lease time 1d
  DHCP, sockets bound exclusively to interface wlan0
  using nameserver 8.8.8.8#53
```

4. Routing und NAT

4.1 IP-Forwarding aktivieren

Damit der Raspberry Pi Pakete zwischen wlan0 und eth0 weiterleitet, muss das IP-Forwarding im Linux-Kernel dauerhaft aktiviert werden:

```
sudo nano /etc/sysctl.conf

# IPv4-Forwarding aktivieren  (Kommentarzeichen entfernen):
net.ipv4.ip_forward=1
```

Änderung sofort übernehmen (ohne Neustart):

```
sudo sysctl -p
```

Verifikation:

```
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

4.2 Routing-Tabelle

Der Linux-Kernel befüllt die Routing-Tabelle automatisch, sobald die Interfaces konfiguriert sind:

```
$ ip route
default via 192.168.178.1 dev eth0 proto dhcp src 192.168.178.69 metric 100
192.168.50.0/24 dev wlan0 proto kernel scope link src 192.168.50.1 metric 600
192.168.178.0/24 dev eth0 proto kernel scope link src 192.168.178.69 metric 100
```

Eintrag	Bedeutung
default via 192.168.178.1 dev eth0	Standard-Gateway: unbekannte Ziele → Heimrouter → Internet
192.168.50.0/24 dev wlan0	WLAN-Subnetz direkt erreichbar über wlan0 (AP-IP: 192.168.50.1)
192.168.178.0/24 dev eth0	Ethernet-Subnetz direkt erreichbar über eth0 (IP: 192.168.178.69)

4.3 NAT / Masquerading

NAT (Network Address Translation) ermöglicht Internet-Zugriff für alle WLAN-Clients: Ausgehende Pakete erhalten die IP-Adresse des eth0-Interfaces als Quelladresse. Die Konfiguration erfolgt in der nftables-NAT-Tabelle (→ Abschnitt 5.2).

```
# postrouting-Chain in nftables:
oif "eth0" masquerade
```

> ***i*** *Masquerading: Alle Pakete von 192.168.50.x erhalten beim Verlassen über eth0 die IP 192.168.178.69. Antwortpakete werden anhand der NAT-Tabelle automatisch zurück an den richtigen WLAN-Client geleitet.*

5. Firewall (nftables)

5.1 Sicherheitskonzept

Die Firewall basiert auf nftables und folgt den fundamentalen Sicherheitsprinzipien aus der Vorlesung 2:

Sicherheitsprinzip	Umsetzung in diesem Projekt
Security by Design	Firewall von Beginn an als Teil der Architektur geplant, nicht nachträglich hinzugefügt
Fail-Secure / Default-Deny	policy drop in input- und forward-Chain: Alles Nicht-Erlaubte wird verworfen
Least Privilege	Nur zwingend notwendige Ports freigegeben: SSH (22), DHCP (67), DNS (53), ICMP
Complete Mediation	Jedes eingehende und weiterzuleitende Paket wird durch nftables geprüft
Least Common Mechanism	Routing ausschließlich wlan0 → eth0 erlaubt; umgekehrte Richtung gesperrt
Defence in Depth	Kombination aus Firewall-Regeln, NAT und Netzwerksegmentierung

5.2 Vollständige nftables-Konfiguration (/etc/nftables.conf)

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {

    chain input {
        type filter hook input priority 0;
        policy drop;                        # Default-Deny

        iif "lo" accept                     # Loopback immer erlauben
        ct state established,related accept   # Stateful: bestehende Verbindungen
        iif "wlan0" tcp dport 22 accept      # SSH aus dem WLAN-Netz
        iif "eth0"  tcp dport 22 accept      # SSH über Ethernet [eigene Erweiterung]
        udp dport 67 accept                  # DHCP-Server
        udp dport 53 accept                  # DNS (UDP)
        tcp dport 53 accept                  # DNS (TCP)
        ip protocol icmp accept              # Ping / ICMP
        log prefix "Dropped packet: " flags all drop  # Logging + Drop
    }
```

```
    chain output {
        type filter hook output priority 0;
        policy accept;                      # Ausgehend alles erlaubt
    }

    chain forward {
        type filter hook forward priority 0;
        policy drop;                        # Default-Deny

        iif "wlan0" oif "eth0" accept       # WLAN → Internet erlaubt
        ct state established,related accept   # Antwortpakete erlaubt
    }
}

table ip nat {
    chain prerouting {
        type nat hook prerouting priority dstnat;
        policy accept;
    }
    chain postrouting {
        type nat hook postrouting priority srcnat;
        policy accept;
        oif "eth0" masquerade               # NAT für WLAN-Clients
    }
}
```

5.3 Erklärung der Firewall-Regeln

Regel	Erklärung
policy drop	Default-Deny: Kein Paket passiert, außer es trifft eine der folgenden Erlaubnisregeln (Fail-Secure).
iif "lo" accept	Loopback-Interface: Interne Prozesse (z. B. DNS-Cache) kommunizieren hierüber. Pflicht-Regel.
ct state established,related accept	Stateful Firewall: Antwortpakete bestehender Verbindungen werden durchgelassen (sonst wäre z. B. HTTP-Response blockiert).
iif "wlan0" tcp dport 22 accept	SSH aus dem WLAN: Verwaltung des Routers von verbundenen IoT-Geräten oder Entwicklungsrechnern im WLAN.
iif "eth0" tcp dport 22 accept	SSH über Ethernet (eigene Erweiterung – siehe 5.4).
udp dport 67 accept	DHCP: Neue WLAN-Clients senden DHCP-Discover auf Port 67 (UDP Broadcast). Ohne diese Regel keine IP-Vergabe.
udp/tcp dport 53 accept	DNS: dnsmasq beantwortet Namensauflösungsanfragen der WLAN-Clients.
ip protocol icmp accept	ICMP / Ping: Erreichbarkeitstests zum und vom Router.
log prefix ... drop	Logging: Verworfene Pakete werden im systemd-Journal protokolliert – wichtig für Debugging und Sicherheitsüberwachung.
iif "wlan0" oif "eth0" accept	Forward WLAN→LAN: Pakete aus dem IoT-Netz dürfen ins Ethernet-Netz weitergeleitet werden (Internetzugriff). Umgekehrte Richtung ist gesperrt.
oif "eth0" masquerade	NAT: Quelladresse aller ausgehenden Pakete wird auf die eth0-IP (192.168.178.69) umgeschrieben.

5.4 Begründung der eigenen Erweiterung: SSH über eth0

In der Musterlösung des Professors ist SSH ausschließlich über wlan0 erlaubt. Die folgende Regel wurde bewusst ergänzt:

```
iif "eth0" tcp dport 22 accept
```

Begründung:

- Der Raspberry Pi ist zu Hause über ein Ethernet-Kabel mit dem Heimrouter verbunden. Ohne diese Regel ist SSH-Zugriff nur möglich, wenn man mit dem WLAN "Production" verbunden ist.
- Während der Entwicklungs- und Konfigurationsphase war es notwendig, den Router direkt über das Heimnetzwerk (192.168.178.0/24) vom Laptop aus per SSH zu verwalten – z. B. bevor der WLAN-AP vollständig eingerichtet war.
- Diese Erweiterung ermöglicht Fernwartung über das heimische Internet-Gateway, ohne sich zuerst mit dem Produktions-WLAN verbinden zu müssen.

> ***i*** *Sicherheitshinweis: In einem echten Produktionsnetzwerk sollte diese Regel durch eine Quelladresseneinschränkung ergänzt werden: iif "eth0" ip saddr 192.168.178.0/24 tcp dport 22 accept – so ist SSH über Ethernet nur aus dem Heimnetz erlaubt, nicht aus dem Internet.*

5.5 Aktivierung und Systemstart

Syntaxprüfung:

```
sudo nft -c -f /etc/nftables.conf
```

Dienst aktivieren und laden:

```
sudo systemctl enable nftables
sudo systemctl restart nftables
```

Firewall startet nach dem Netzwerk (systemd-Override):

```
sudo systemctl edit nftables

[Unit]
After=network-online.target
Wants=network-online.target
```

Aktive Regeln anzeigen:

```
sudo nft list ruleset
```

Verworfene Pakete im Log:

```
sudo journalctl -k -n 80 | grep "Dropped packet"
```

6. Konfigurationsnachweis – Systemverifikation

Die folgenden Ausgaben wurden direkt vom System abgerufen und belegen die korrekte Konfiguration aller Komponenten.

6.1 IP-Forwarding

```
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

6.2 Aktives Firewall-Regelwerk

```
$ sudo nft list ruleset
table inet filter {
  chain input  { type filter hook input priority filter; policy drop;
    iif "lo" accept  |  ct state established,related accept
    iif "wlan0" tcp dport 22 accept  |  iif "eth0" tcp dport 22 accept
    udp dport 67 accept  |  udp dport 53 accept  |  tcp dport 53 accept
    ip protocol icmp accept
    log prefix "Dropped packet: " flags all drop  }
  chain output  { policy accept; }
  chain forward { policy drop;
    iif "wlan0" oif "eth0" accept  |  ct state established,related accept  }
}
table ip nat {
  chain postrouting { oif "eth0" masquerade }
}
```

6.3 dnsmasq – DHCP & DNS

```
$ sudo systemctl status dnsmasq
● dnsmasq.service  Active: active (running) since Sat 2026-04-18 23:51:21 CEST; 13h ago
  DHCP, IP range 192.168.50.20 -- 192.168.50.150, lease time 1d
  DHCP, sockets bound exclusively to interface wlan0
  using nameserver 8.8.8.8#53  /  using nameserver 192.168.178.1#53
  using only locally-known addresses for lab.local
```

6.4 Routing-Tabelle

```
$ ip route
default via 192.168.178.1 dev eth0 proto dhcp src 192.168.178.69 metric 100
192.168.50.0/24 dev wlan0 proto kernel scope link src 192.168.50.1 metric 600
192.168.178.0/24 dev eth0 proto kernel scope link src 192.168.178.69 metric 100
```

6.5 Gesamtstatus

Komponente	Status	Verifikationsbefehl
WLAN-AP (wlan0)	Aktiv	nmcli device status
DHCP (dnsmasq)	Aktiv	systemctl status dnsmasq
DNS lokal (lab.local)	Aktiv	systemctl status dnsmasq
IP-Forwarding	Aktiv (= 1)	sysctl net.ipv4.ip_forward
Routing-Tabelle	Korrekt	ip route
Firewall Input	Default-Deny	nft list ruleset
Firewall Forward	wlan0 → eth0	nft list ruleset
NAT / Masquerading	Aktiv	nft list table ip nat
SSH (wlan0)	Erlaubt	nft list ruleset
SSH (eth0 – Erweiterung)	Erlaubt	nft list ruleset

7. Testszenarien

Die folgenden Tests verifizieren alle Komponenten systematisch.

7.1 WLAN-Access-Point

Test	Befehl	Erwartetes Ergebnis
AP-Status	sudo nmcli device status	wlan0: wifi, verbunden
Verbundene Clients	sudo iw dev wlan0 station dump	MAC, Signal, TX-Rate der Clients
SSID sichtbar	sudo iw dev wlan0 scan | grep SSID	"Production" erscheint
DHCP-Lease am Client	ip a show wlan0	IP aus 192.168.50.20–.150
Lease-Liste	cat /var/lib/misc/dnsmasq.leases	MAC, IP, Hostname der Clients

7.2 Routing & Konnektivität

Test	Befehl	Erwartetes Ergebnis
IP-Forwarding	sysctl net.ipv4.ip_forward	= 1
Routing-Tabelle	ip route	3 Einträge: default, wlan0, eth0
Ping Gateway	ping -c 4 192.168.50.1	0 % Paketverlust
DNS-Auflösung	ping -c 4 www.google.com	Antwort von Google
Traceroute	traceroute www.google.com	1. Hop: 192.168.50.1, 2.: 192.168.178.1
NAT-Test	curl -s https://api.ipify.org	Öffentliche IP des Heimanschlusses

7.3 Firewall

Test	Befehl / Aktion	Erwartetes Ergebnis
Regelwerk laden	sudo nft list ruleset	Vollständige Tabellen sichtbar
SSH aus WLAN (soll klappen)	ssh projekt_user@192.168.50.1	Verbindung aufgebaut
SSH über Ethernet (soll klappen)	ssh projekt_user@192.168.178.69	Verbindung aufgebaut
Gesperrter Port (soll scheitern)	nc -zv 192.168.50.1 80	Connection refused / Timeout
Drop-Logging	journalctl -k | grep 'Dropped packet'	Log-Einträge erscheinen
Dienst aktiviert	systemctl is-enabled nftables	enabled

8. Python-Skript zur automatischen Konfiguration

Gemäß der Aufgabenstellung wurde ein Python-Programm entwickelt, das die gesamte Konfiguration des WLAN-AP-Routers automatisch durchführt.

Das Skript ist bewusst in eine modulare Paketstruktur aufgeteilt, um Konfiguration, Logik und Dienste voneinander zu trennen:

```
wlan-ap-router/
├── main.py          # Einstiegspunkt
├── config/          # settings.py, paths.py  (Konstanten)
├── configs/         # dnsmasq.conf, nftables.conf, sysctl_append.conf
├── core/            # utils.py, installer.py, cleanup.py
└── services/        # access_point.py, dnsmasq.py, firewall.py, …
```

Damit Python die Ordner `core/`, `services/` und `config/` als importierbare Pakete erkennt, enthält jeder dieser Ordner eine leere (bzw. mit Export-Anweisungen versehene) Datei `__init__.py` – ohne diese Datei würde `from core import install_packages` in `main.py` mit einem `ModuleNotFoundError` scheitern.

> ***i*** *Abweichung von der Musterlösung (Vorlesung Folie 2-50): In der Vorlesung wird als ausgehendes Interface `oif "wlan1"` für das Masquerading verwendet. In diesem Projekt existiert kein zweites WLAN-Interface – der Raspberry Pi ist per Ethernet ans LAN angebunden. Daher wurde korrekt `oif "eth0"` eingesetzt. Dies entspricht dem tatsächlichen Hardware-Setup und ist durch die Routing-Tabelle und den Verifikationsnachweis in Abschnitt 6 belegt.*

8.1 Aufgaben des Skripts

- Pakete installieren (hostapd, dnsmasq)
- WLAN-AP via nmcli anlegen und Passwort setzen
- dnsmasq konfigurieren (DHCP, DNS, lab.local)
- IP-Forwarding dauerhaft aktivieren (/etc/sysctl.conf)
- nftables-Firewall-Konfiguration schreiben (/etc/nftables.conf)
- Alle Dienste aktivieren und starten (systemd)
- Abschließende Verifikation aller Komponenten

8.2 Ausführung

```
sudo python3 wlan_ap_setup.py
```

8.3 Skript-Auszug: Kernfunktionen

```
def install_packages()          → apt install hostapd dnsmasq
def configure_access_point()    → nmcli: AP anlegen + Passwort
def configure_dnsmasq()         → /etc/dnsmasq.conf schreiben
def enable_ip_forwarding()      → /etc/sysctl.conf + sysctl -p
def configure_firewall()        → /etc/nftables.conf schreiben
def enable_services()           → systemctl enable/restart
def verify_configuration()      → Status aller Komponenten prüfen
def main()                      → Orchestrierung aller Schritte
```

9. Begriffsdefinitionen

Begriff	Erklärung
WLAN / WiFi	Wireless Local Area Network nach IEEE 802.11
SSID	Service Set Identifier – Name des WLAN-Netzwerks
WPA2	Wi-Fi Protected Access 2 – Verschlüsselungsstandard
Access Point	Gerät, das eine WLAN-Funkzelle bereitstellt
DHCP	Dynamic Host Configuration Protocol – automatische IP-Vergabe
DNS	Domain Name System – Hostname → IP-Auflösung
dnsmasq	Leichtgewichtiger DNS/DHCP-Server für kleine Netzwerke
NAT / Masquerade	Network Address Translation – private IP → öffentliche IP
nftables	Modernes Linux-Paketfilter-Framework (Nachfolger von iptables)
Default-Deny	Sicherheitsprinzip: alles verboten außer explizit erlaubt
IP-Forwarding	Kernel-Funktion zur Paketweiterleitung zwischen Interfaces
mac80211	Linux-Kernel-Subsystem für IEEE 802.11 WLAN
hostapd	Host AP Daemon – WPA2/WPA3-Authentifizierung
NetworkManager	Linux-Dienst zur Verwaltung von Netzwerkverbindungen
ct state	Connection Tracking – Zustandsverfolgung von Verbindungen
iif / oif	incoming / outgoing interface in nftables-Regeln
Stateful Firewall	Firewall, die den Verbindungsstatus verfolgt (ct state)
Security by Design	Sicherheit von Anfang an in die Architektur eingebaut